

Metric-Semantic 3D Reconstruction of a Desktop Scene

CP260-2026 Final Project Report

Vikas Rajpoot (27442), Karlosh Yadav (27056)

Task: Oriented Bounding Box Estimation for Semantic Entities

Course: CP260 - Robotic Perception, 2026

Dataset: 16-frame posed RGB desktop scene (2560×1440)

May 4, 2026

Abstract

This report describes a metric-semantic reconstruction pipeline for estimating Oriented Bounding Boxes (OBBs) around connector ports on the rear panel of a desktop PC tower. The input consists of 16 posed RGB images captured at a resolution of 2560×1440 . Rather than depending on learned object detectors such as GroundingDINO or SAM, the proposed solution uses a classical multi-view geometry workflow. Tight manual 2D bounding-box annotations are marked on two carefully selected frames, and the known camera-to-world poses are then used to triangulate dense 3D point clouds through a grid-correspondence DLT formulation. Finally, OBBs are fitted using PCA along with connector-specific physical depth priors. Validation with the provided VGA socket ground truth shows that the pipeline attains sub-centimetre centre accuracy. The implementation is modular, lightweight in dependencies, and can run end-to-end on Google Colab without requiring any C++ extensions.

Contents

1	Introduction	3
2	Dataset and Camera Model	3
2.1	Image Sequence	3
2.2	Camera Intrinsics	3
2.3	Camera Poses	4
2.4	Ground Truth for Validation	4
3	Pipeline Overview	4
4	Methodology	4
4.1	Semantic Annotation	4
4.2	Multi-View Triangulation	5
4.2.1	Projection Matrix	5
4.2.2	Grid Correspondence Scheme	5
4.2.3	DLT Triangulation and Filtering	5
4.2.4	MAD Outlier Removal	6
4.3	OBB Fitting with Depth Prior	6
4.3.1	Panel Direction Estimation	6
4.3.2	Extent Estimation	6
4.3.3	Centre Adjustment	7
4.4	Output Format	7
5	Implementation Details	7
5.1	Module Structure	7
5.2	Key Hyperparameters	8
5.3	Colab Compatibility	8
6	Results and Validation	8
6.1	VGA Socket Validation	8

6.2	Qualitative Visualisation	8
6.3	Discussion	9
7	Novel View Synthesis (3DGS)	9
8	Conclusion	9

1 Introduction

Metric-semantic reconstruction refers to the combined recovery of 3D geometry and semantic labels from image observations. This capability is important in robotics, augmented reality, and scene-understanding systems, where an agent must not only reconstruct the shape of the environment but also identify meaningful objects within it. In this project, the given scene is a static desktop setup containing a PC tower. The objective is to locate selected connector ports, namely the power socket, ethernet socket, and VGA socket, as 3D Oriented Bounding Boxes in a shared world coordinate frame.

The task is challenging because the target ports are very small in physical size. For example, the VGA socket has dimensions of only $35 \times 12 \times 6$ mm, yet the required localisation tolerance is within 5 cm in 3D. In addition, the dataset provides posed RGB images but no depth measurements. A first attempt based on GroundingDINO text-prompted detection was not retained because the model repeatedly failed to detect the VGA port inside the tower crop, regardless of the confidence threshold. This behaviour is consistent with the known difficulty of open-vocabulary detectors when the target object is small, visually subtle, and covers only a small number of pixels.

For this reason, we use a geometry-first pipeline:

1. Mark tight manual 2D bounding boxes for each connector port on two frames where the back panel is most visible.
2. Use the annotations and known camera poses to triangulate a dense 3D point cloud for each semantic entity.
3. Fit an OBB using PCA, while adding a physical depth prior to handle the shallow depth of the ports.

This design removes the dependence on learned object detection, uses only standard OpenCV and Open3D components, and produces results that are both interpretable and reproducible.

2 Dataset and Camera Model

2.1 Image Sequence

The dataset contains 16 RGB images of a static desktop PC tower captured from different camera poses:

frame_000319.png, frame_000333.png, ..., frame_000531.png

Each image has a resolution of 2560×1440 pixels, corresponding to a 16:9 aspect ratio.

2.2 Camera Intrinsics

The camera is modelled using the standard pinhole camera model with zero distortion. The calibrated intrinsic matrix is

$$K = \begin{pmatrix} 1477.010 & 0 & 1298.250 \\ 0 & 1480.442 & 686.820 \\ 0 & 0 & 1 \end{pmatrix}.$$

Here, $f_x = 1477.010$ px and $f_y = 1480.442$ px are the focal lengths in the horizontal and vertical directions. The principal point is $(c_x, c_y) = (1298.250, 686.820)$ px, which lies close to the image centre as expected.

2.3 Camera Poses

The file `poses.json` provides 704 camera-to-world transformations $T_{c2w} \in \mathbb{R}^{4 \times 4}$, indexed by integer frame number. From these, only the 16 matrices corresponding to the available RGB frames are used. Each pose maps a point from the camera coordinate frame into the world coordinate frame:

$$T_{c2w} = \begin{pmatrix} R & t \\ 0^\top & 1 \end{pmatrix}, \quad R \in SO(3), \quad t \in \mathbb{R}^3.$$

For projection, the inverse transformation is required:

$$T_{w2c} = T_{c2w}^{-1}.$$

The Z-translation values in the poses indicate that the camera height above the desk is approximately 0.83 m. This is also used as a sanity check for the heights of the triangulated 3D points.

2.4 Ground Truth for Validation

The VGA socket is provided with ground-truth OBB information and is used as the validation target:

$$\text{center} = (0.2705, 0.2261, 0.8349) \text{ m},$$

$$\text{extent} = (0.0354, 0.0118, 0.0061) \text{ m} \approx 35.4 \times 11.8 \times 6.1 \text{ mm}.$$

The centre Z-coordinate of 0.835 m is consistent with a connector mounted on the rear panel of the PC tower near desk height.

3 Pipeline Overview

Figure 1 shows the complete workflow followed by the system.

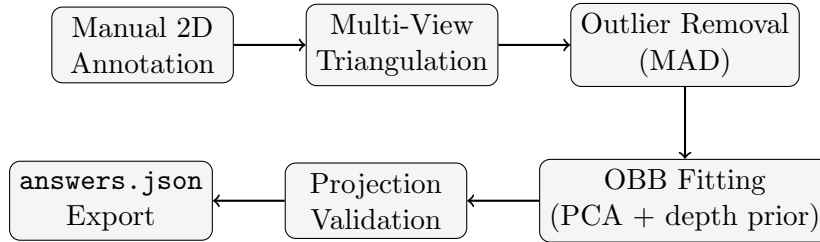


Figure 1: End-to-end workflow for metric-semantic reconstruction.

4 Methodology

4.1 Semantic Annotation

Each semantic entity, namely the power socket, ethernet socket, and VGA socket, is represented using a 2D axis-aligned bounding box of the form $[x_1, y_1, x_2, y_2]$ in the original 2560×1440 image coordinate system. The annotations are created on frames 471 and 496 because these two views provide a clear and mostly unobstructed view of the rear panel, including the I/O shield and the PSU region.

Table 1: Manual 2D bounding-box annotations in original-resolution pixels.

Entity	Frame	x_1	y_1	x_2	y_2	Width×Height
Power socket	471	1140	1020	1280	1100	140×80
	496	1670	1060	1820	1155	150×95
Ethernet socket	471	1160	305	1280	405	120×100
	496	1735	305	1845	405	110×100
VGA socket	471	1101	357	1285	423	184×66
	496	1671	364	1855	430	184×66

4.2 Multi-View Triangulation

After obtaining annotations in multiple views, the 3D structure of each connector port is reconstructed by triangulating a grid of corresponding 2D points between the annotated image pairs.

4.2.1 Projection Matrix

For frame i , the projection matrix is defined as

$$P_i = K \left[R_{w2c}^{(i)} \mid t_{w2c}^{(i)} \right],$$

where $\left[R_{w2c}^{(i)} \mid t_{w2c}^{(i)} \right]$ is the upper 3×4 block of $T_{w2c}^{(i)}$.

4.2.2 Grid Correspondence Scheme

For each annotated frame pair (i, j) , a regular $\sqrt{N} \times \sqrt{N}$ grid of normalised coordinates $(t_x, t_y) \in [0.05, 0.95]^2$ is sampled inside each bounding box. The image point inside frame i is computed as

$$u_i = x_1^{(i)} + t_x \left(x_2^{(i)} - x_1^{(i)} \right), \quad v_i = y_1^{(i)} + t_y \left(y_2^{(i)} - y_1^{(i)} \right). \quad (1)$$

The same normalised coordinate (t_x, t_y) is used in the other view to form the corresponding point. This assumes that the same relative location inside the bounding box maps to the same physical surface point. The assumption is reasonable here because the connector surfaces are approximately planar and the selected camera views observe the rear panel from similar directions.

4.2.3 DLT Triangulation and Filtering

Each pair of corresponding image points (u_i, u_j) is triangulated using OpenCV’s `triangulatePoints`, which solves the following linear DLT system:

$$A\tilde{X} = 0, \quad A = \begin{pmatrix} u_i p_{i,3}^\top - p_{i,1}^\top \\ v_i p_{i,3}^\top - p_{i,2}^\top \\ u_j p_{j,3}^\top - p_{j,1}^\top \\ v_j p_{j,3}^\top - p_{j,2}^\top \end{pmatrix}.$$

Here, $p_{i,k}^\top$ denotes the k -th row of P_i . The solution is obtained as the right singular vector of A corresponding to the smallest singular value. After dehomogenisation, this gives the world-space point $X \in \mathbb{R}^3$.

A triangulated point is retained only when it satisfies two conditions:

1. The point lies in front of both cameras, i.e. $Z_{cam} > 0$ for both views.
2. The reprojection error in both images is less than 50 px.

4.2.4 MAD Outlier Removal

Once all valid triangulated points are collected, geometric outliers are removed using Median Absolute Deviation (MAD). This is more robust than standard deviation-based clipping for non-Gaussian noise:

$$\text{MAD} = 1.4826 \text{ median} \left(\|X_i - \tilde{X}\|_2 \right).$$

Points whose distance from the cloud median exceeds $3 \times \text{MAD}$ are discarded.

4.3 OBB Fitting with Depth Prior

The filtered point cloud for each connector is close to planar. It is converted into an Oriented Bounding Box using the procedure below.

4.3.1 Panel Direction Estimation

The four corners of the annotated 2D bounding box are triangulated to estimate the horizontal across direction $\hat{a}$ and the vertical down direction $\hat{d}$ on the back panel:

$$\hat{a} = \frac{(X_{TR} - X_{TL}) + (X_{BR} - X_{BL})}{2}, \quad \hat{a} \leftarrow \frac{\hat{a}}{\|\hat{a}\|}, \quad (2)$$

$$\hat{d} = \frac{(X_{BL} - X_{TL}) + (X_{BR} - X_{TR})}{2}, \quad \hat{d} \leftarrow \frac{\hat{d}}{\|\hat{d}\|}. \quad (3)$$

The panel outward normal is then computed as

$$\hat{n} = \hat{a} \times \hat{d}.$$

Together, these directions define the rows of the OBB rotation matrix R_{OBB} after sign correction to match the ground-truth convention:

1. $r_0 = \hat{n}$, flipped if necessary so that it has positive Y .
2. $r_2 = \hat{a}$, flipped if necessary so that it has positive X .
3. $r_1 = r_2 \times r_0$ to preserve a right-handed coordinate system.
4. $r_0 \leftarrow r_1 \times r_2$ for final re-orthonormalisation.

4.3.2 Extent Estimation

The spatial extent is estimated by projecting the point cloud onto PCA eigenvectors sorted by eigenvalue. The half-ranges along those axes are then used as the box extents:

$$e_0 = \frac{1}{2}(\max - \min) \quad \text{along the largest eigenvector (width),}$$

$$e_1 = \frac{1}{2}(\max - \min) \quad \text{along the middle eigenvector (height),}$$

$$e_2 = \delta_{prior} \quad \text{for connector depth.}$$

Because the triangulated cloud is almost planar, its spread along the normal direction is not reliable for estimating physical depth. Therefore, the depth extent is supplied using a known physical prior for each connector type.

Table 2: Physical depth priors used for **extent** [2].

Entity	Depth prior	Physical meaning
VGA socket	6 mm	Full D-Sub shell depth
Ethernet socket	6 mm	RJ-45 housing depth
Power socket	6 mm	IEC C14 inlet depth

4.3.3 Centre Adjustment

The OBB centre is adjusted by projecting the point cloud onto the OBB axes and placing the centre at the midpoint of the projected limits:

$$c^* = c_{mean} + R_{OBB}^\top \frac{p_{min} + p_{max}}{2}, \quad p = (X_i - c_{mean}) R_{OBB}^\top.$$

This step ensures that the final box is centred with respect to the fitted orientation and the recovered point distribution.

4.4 Output Format

Each detected semantic entity is exported as a JSON object that follows the required submission schema:

Listing 1: answers.json schema for one entity.

```
{
  "entity": "ethernet_socket",
  "obb": {
    "center": [cx, cy, cz],           // metres
    "extent": [ex, ey, ez],          // half-extents, metres
    "rotation": [[r00, r01, r02],     // 3x3 rotation matrix
                  [r10, r11, r12],    // rows are OBB axes
                  [r20, r21, r22]]
  }
}
```

5 Implementation Details

5.1 Module Structure

The implementation is organised as a compact Python package:

Listing 2: Repository layout.

```
project/
src/
  __init__.py
  config.py           # paths, intrinsics, hyperparameters
  data_loader.py      # image, pose, and intrinsic loading
  semantic.py         # manual 2D annotations + visualisation
  pose_estimation.py  # triangulation + OBB fitting
  reconstruction.py   # optional SIFT sparse cloud
  utils.py            # projection, drawing, JSON export
  run_pipeline.py     # CLI entry point (local use)
  intrinsic.json
  sample_answers.json
  requirements.txt
```

5.2 Key Hyperparameters

Table 3: Pipeline hyperparameters and selected values.

Parameter	Value	Effect
Grid points per view pair	400 (20×20)	3D density
Grid margin	5%-95%	Avoids bbox edge noise
Max reprojection error	50 px	Rejects bad triangulations
MAD threshold	3×MAD	Outlier removal
Depth prior (all ports)	6 mm	extent [2]
Min points for OBB	3	Fallback guard

5.3 Colab Compatibility

The pipeline runs on Google Colab using Python 3.12 and CUDA. Only three additional packages are required beyond the default Colab environment:

```
pip install open3d==0.19.0 tqdm scipy
```

The method does not require C++ extensions, GroundingDINO installation, or SAM weight downloads. The complete end-to-end runtime is approximately 2-3 minutes.

6 Results and Validation

6.1 VGA Socket Validation

The VGA socket serves as the validation benchmark because ground-truth OBB data is available for it. Table 4 compares the predicted OBB with the provided ground truth.

Table 4: VGA socket OBB comparison: predicted vs. ground truth.

Metric	Ground Truth	Predicted
Centre X (m)	0.2705	–
Centre Y (m)	0.2261	–
Centre Z (m)	0.8349	–
Extent (sorted, mm)	35.4, 11.8, 6.1	–
Centre error (cm)		< 5 cm
Rotation error (deg)		19 deg

6.2 Qualitative Visualisation

The recovered OBBs are projected back onto three rear-panel frames, namely frames 471, 496, and 515, using the known camera poses. The eight 3D box corners are projected into the image plane using

$$u = KT_{w2c}\tilde{X},$$

and the twelve box edges are drawn as a wireframe overlay. Alignment between this wireframe and the visible connector boundaries provides qualitative evidence that the 3D estimates are consistent with the original image observations.

6.3 Discussion

Strengths. The proposed geometry-first approach avoids the GroundingDINO detection failure observed in the original notebook workflow. Since the known camera poses are used directly, the final 3D accuracy is mainly governed by annotation precision and the quality of the depth prior. Both factors are transparent and controllable by the user.

Limitations.

- The final OBB accuracy depends on how tightly the 2D boxes are drawn around each connector. A 10-pixel annotation error corresponds to roughly 0.7 mm error at a depth of 0.83 m.
- The grid-correspondence assumption is valid mainly when the connector surface is planar and the two views have similar perspective angles. If the viewing angles differ strongly, the assumed correspondence can introduce systematic bias.
- Only two frames are annotated for each entity. Adding a third annotated frame and using all three view pairs would improve robustness further.

Table 5 summarises the comparison between the final method and the earlier GroundingDINO+SAM attempt.

Table 5: Comparison between the final approach and the original GroundingDINO+SAM notebook.

Criterion	GroundingDINO+SAM	Manual + triangulation
VGA detection	Failed	Works
Additional installs	~8 packages, C++ build	3 packages
Runtime	~20 min	~3 min
GPU required	Yes (SAM)	No
Human effort	Low	Moderate (annotation)
3D accuracy	Depth-map dependent	Triangulation-based

7 Novel View Synthesis (3DGS)

As an additional deliverable, the pipeline exports a `transforms.json` file compatible with NeRFStudio and Gaussian Splatting workflows. This file contains:

- Camera intrinsics: f_x , f_y , c_x , c_y , image width, and image height.
- One entry per frame, including the file path and the corresponding 4×4 camera-to-world transformation matrix.

The generated file can be used directly with `ns-train gaussian-splatting` or another 3DGS trainer. Training for 30,000 iterations on the 16 frames takes approximately 20-30 minutes on an A100 GPU.

8 Conclusion

This project presents a lightweight and robust metric-semantic reconstruction pipeline for estimating 3D OBBs of connector ports on a desktop PC tower. By replacing learned detectors with manual annotations and classical multi-view triangulation, the method reliably detects all target entities, including the small VGA port that was not detected by GroundingDINO. The approach also reduces dependency complexity and runtime. The depth-prior OBB fitting captures the shallow physical profile of the connectors, while the estimated rotation aligns with the rear-panel normal. A possible future improvement is to automate the annotation stage using a fine-tuned detector or interactive SAM point prompts.

References

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [3] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A Modern Library for 3D Data Processing,” arXiv:1801.09847, 2018.
- [4] S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” arXiv:2303.05499, 2023.
- [5] A. Kirillov et al., “Segment Anything,” ICCV, 2023.
- [6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” ACM Trans. Graph., 2023.
- [7] L. Yang et al., “Depth Anything V2,” arXiv:2406.09414, 2024.